IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 23 November 2024, accepted 2 March 2025, date of publication 7 March 2025, date of current version 14 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3549183

RESEARCH ARTICLE

Empirical Investigation of Key Enablers for Secure DevOps Practices

MUHAMMAD AZEEM AKBAR[1] **AND ABEER ABDULAZIZ ALSANAD**[2]
[1]Software Engineering Department, Lappeenranta–Lahti University of Technology, 53851 Lappeenranta, Finland
[2]Information Systems Department, Imam Mohammad Ibn Saud Islamic University, Riyadh 11432, Saudi Arabia

Corresponding author: Muhammad Azeem Akbar (azeem.akbar@lut.fi)

ABSTRACT DevSecOps integrates security practices from development to operations in the search for faster and more secure responses to business needs. DevSecOps is an approach based on the 4 pillars of the CAMS model (culture, measurement, automation, and sharing). On the other hand, there is an increasing interest in ensuring sustainability in software work making it more optimal and viable overtime. Given that sustainability has not received much attention in the DevSecOps arena, this study aims to identify the enablers that are important for addressing CAMS by means of a multivocal literature review and to check their practical implacability in real-world practices. To do so, firstly, 35 enablers related to CAMS principles were identified as a result of the multivocal literature review conducted. Based on the literature findings, a hypothetical model was developed based on the DevSecOps enablers. Next, a quantitative survey was executed, and 214 responses were collected from experts. Finally, authors used Smart-PLS (3.0) and analyzed collected data to form the final set of enablers. Results show that these enablers could positively impact the execution of DevSecOps practices in a sustainable way. Additionally, results also reveal that when deploying DevSecOps, practitioners should consider the enablers that were explored and mapped against culture, automation, measurement, and sharing. The findings of this study will help practitioners and academics to understand DevSecOps principles and the areas that need to be considered by industry professionals to make DevSecOps sustainable.

INDEX TERMS DevSecOps, CAMS, enablers, empirical investigations.

I. INTRODUCTION

Sustainable software development is a set of practices that enable a team to achieve and maintain an optimal development pace indefinitely [1], [2]. It is becoming a timely needed topic of discussion in the information technology domain due to the software industry's growing carbon footprint and increasing social responsibility in other business world [3]. Various businesses and everyday life activities are becoming more dependent on software systems, which overall increases the software industry ethical responsibilities [4]. Sustainable software development should not only consider a technical and economical requirement but also the fact that practitioners take ownership of long-term customer satisfaction. Managing customer requirements, business needs, and features prioritization are the most challenging sustainable software development factors [5]. According to Penzenstadler [5], the general definition of software engineering sustainability is "the capacity to endure" and sustainable development refers to *"meeting the needs of the present without compromising the ability of future generations to meet their own needs."*

Cultural sustainability has become a growing priority within sustainable development agendas [6]. This renders that software development culture should be based on close collaboration and communication between team members and product customers. The best practice to develop such working culture could be achieved by adopting continuous

The associate editor coordinating the review of this manuscript and approving it for publication was Antonio Piccinno.

software development practices, which promotes communication between individuals and software technical teams [7]. Continuous software engineering merges the development and operations (DevOps) practices, which have been widely adopted and considered a modern software development approach. According to Leite et al. [8], *"DevOps is a collaborative and multidisciplinary effort within an organization to automate continuous delivery of new software versions while guaranteeing their correctness and reliability."*

DevOps focuses on continuous development and operations that encapsulate both development and operation teams under a single umbrella [9]. DevOps practices encourage the development and operation teams to work in a collaborative culture to faster software development, operations, and release activities. In DevOps, development and operation teams as well as customers and quality assurance stakeholders collaborate to deliver a software product continuously, take market opportunities, and reduce the customer feedback time [10], [11].

DevOps practices are continuously evolving and, in the current era, the concept of DevSecOps (development, security, and operations) has been presented, adding security concerns to current DevOps approaches [12]. DevSecOps encompasses not only Development (Dev) and Operations (Ops) teams, but also Security (Sec) teams to ensure the developing and delivering secure software products to customers [13]. Despite the significance of considering security in the DevOps domain, little research has been done to understand the core dimensions of DevSecOps i.e., CAMS's principles (culture, automation, measurement, and sharing) [14], [15], [16]. Therefore, significant research contributions need to focus on core CAMS areas to develop sustainable and secure software systems using DevSecOps practices.

We conducted this study intending to explore the real-world significance of the DevSecOps practices and discuss the fundamental enablers (factors) of CAMS principles. The result of this study presents several novel contributions to the field of DevSecOps by exploring its real-world significance and identifying 35 enablers critical for sustainable implementation, mapped to the CAMS principles: Culture, Automation, Measurement, and Sharing. Using a robust Multivocal Literature Review (MLR) approach, incorporating both primary studies and grey literature, the study provides a comprehensive understanding of these enablers. A hypothetical framework was developed and empirically validated through a large-scale questionnaire survey with 214 industry practitioners, analyzed using Partial Least Squares Structural Equation Modeling (PLS-SEM). This approach ensures theoretical rigor and practical applicability. By emphasizing sustainability—a relatively unexplored aspect in DevSecOps—this research offers actionable guidance for practitioners, advancing both academic and practical understanding of secure and sustainable software development practices. The findings are expected to significantly contribute to making DevSecOps processes successful from multiple perspectives. The following research questions are addressed in this paper:

[RQ1] What enablers of sustainable DevSecOps are reported in the literature concerning to CAMS principles?

[RQ2] What are the practitioner's perceptions regarding the literature-based enablers of sustainable DevSecOps?

The rest of the study is organized as: background is presented in section II. Research design and hypothesis are discussed in section III. Study results and analysis are presented in section IV. Results are discussed in Section V while the limitations and implications of this study are presented in sections VI and VII, respectively. Related work is included in section VIII. Finally, conclusions and future directions are p in section IX.

II. BACKGROUND

It is important to develop and deliver software with high quality and swiftly in the software engineering industry. To achieve such industrial objectives, continuous software engineering is one of the paradigms widely used by software teams. Continuous software engineering aims to develop a consistent software development flow to instantly and continuously deliver software products [17]. Continuous software engineering is based on continuous software development and delivery activities e.g., continuous integration [18], continuous delivery [19], and continuous deployment [20]. Fitzgerald and Stol [17] shed light on different categories of continuous practices under business planning, development, and operations domains. In particular, continuous software engineering is associated with development and operations (DevOps) practices, which have been widely adopted in modern-day software development approaches.

DevOps is a set of practices that merges continuous software engineering activities from development to delivery. DevOps is a natural extension of Agile and continuous delivery methods [21]. It provides practices focused on the organization's culture, where development and operation teams work as one unit, unlike traditional approaches [21]. Such traditional approaches are time-consuming and lack understanding of development and operation silos, which lead to more effort-consuming activities, ultimately ending in customer dissatisfaction [22]. On the other hand, the DevOps processes provide a culture that allows fast, efficient, reliable, and continuous software delivery [23].

Shackleford [24] underlined the complexities of managing organizational changes in terms of tools, standards, and processes for integrating security in DevOps practices. Such a need has led to the coining of the term, DevSecOps. The main objective of DevSecOps is to keep security as a priority and focus on integrating security across phases of DevOps practices. DevSecOps deals with security concerns from design to integration testing and software delivery [25]. Incorporating security in Dev and Ops silos consists of various potential challenges [12]. For instance, lack of secure coding standards,

lack of security feature review, lack of continuous software security awareness, and neglecting change control in security [26]. It means that security approaches need to be highly agile and accepted and understood by security, development, and operations teams [27].

DevSecOps concepts are understandable and can be adopted by industry experts [28], [29], however, it is challenging to envision how to put these concepts into practice [30]. According to the state of DevOps report [28], models such as The Three Ways of DevOps, CAMS, and CALMS (stands for Collaboration, Automation, Lean, Measurement, and Sharing) provide valuable principles for improving the social interactions. Indeed, CAMS has also drawn the attention of researchers [12], [14], [15], [16], [31]. Myrbakken and Colomo-Palacios [12] studied CAMS principles to characterize DevSecOps and explored how these principles could contribute to gaining benefits if they are successfully adopted. Sánchez-Gordón and Colomo-Palacios [14] suggest that CAMS principles, in particular ''culture'', should be considered to cover key aspects of DevSecOps processes. Likewise, Koskinen [16] conducted a literature review to identify how are the CAMS principles reflected in secure DevOps research. Tomas et al. [31] interviewed six practitioners to gain insights into the CAMS principle and security. They concluded that CAMS can provide a solid foundation to adequately integrate security into daily practices but it requires developing a culture concerning DevSecOps, automating wherever possible to eliminate errors, measure and keep track of the releases, and share feedback, ideas, and resources with trusted people [31]. In what follows, each principle that comprises CAMS is described.

Culture: Culture has been reported as the most critical and challenging aspect of the DevSecOps process [14]. The importance of culture should not be underestimated, since culture refers to bridging the communication gap between development, security, and operations teams to work together, as a team, for a common organizational goal [14]. DevSecOps focuses on culture by breaking down silos while sharing responsibilities [32] and increasing explicit collaboration between different roles that improve the overall working environment [31]. Ultimately, a security culture leads to continuous learning and improvement of software development and delivery that shifts security left [32].

Automation: The main objective is to automate software development and release activities to eliminate human errors and vulnerabilities. The set of automated processes and tools to develop, build, test, and deploy software is called a ''pipeline''. In this context, automating repetitive tasks may not be sufficient but it is necessary to improve overall software quality [29]. In fact, automation is fundamental for any successful DevOps initiative, including those focused on security [31]. Automation helps to improve the organization's workflow and productivity to a large extent but also enables practitioners to find the capacity to focus on other tasks that deliver value [28]. Although the pipeline itself can lead to security-related challenges [30], the implementation of automated test that runs at any point in the pipeline can provide continuous and immediate feedback cycles [14].

Measurement: Measurement refers to monitoring and tracking the progress DevSecOps process activities. Measurement is also a significant aspect of DevSecOps, as it helps to know how the system works or what needs to be done to increase security, performance, and productivity [31], [33]. DevSecOps measurement assists in determining the progress being made towards the project's finalization. Generally, two major points need to be considered while using measurement metrics. Firstly, ensure the correctness of the selected measurement metrics. Secondly, incentivize the right measurement metrics [33]. DevSecOps practices emphasize: ''see the forest from the trees'' by examining the entire operational phase and measuring the whole procedure, not just a part [34].

Sharing: Sharing information, tools, and experiences among team members are the keys to DevSecOps success. Spreading knowledge helps develop close feedback loops and allows the organization to improve over time [30], [31]. The team becomes more efficient due to this collective intelligence, and it becomes more than the sum of its parts [30]. Sharing has many benefits inside and outside the organization, i.e., finding people in the organization who have similar requirements will create fresher opportunities to collaborate. Also, redundant work can be eliminated. Finding people with common interests will result in better engagement among them. Outside the organization, sharing resources within trusted communities helps implement new open-source software features more quickly [31].

III. RESEARCH DESIGN

To come up the study objective, firstly, a MLR [35] approach is used to explore existing literature studies in order to identify the core enablers, related to four CAMS principles. Secondly, we proposed a hypothetical framework and developed the four hypotheses, based on MLR findings. In the next phase, the proposed hypotheses were empirically evaluated by collecting data from real-world experts using a questionnaire survey instrument. Both MLR and questionnaire survey approaches are subsequently discussed in the following sections:

A. MLR

The MLR study was conducted to explore the DevSecOps enablers reported in the grey and formal literature. Grey literature refers to magazines, white papers, government reports, videos, blog reports, etc., where the publication is not the primary aim of the publishing body. The formal literature presents the pre-reviewed studies published in the mainstream research bodies. In this review, we considered both types of studies as grey literature give the vibe of industry experts, and formal literature (primary studies) presents the experiences and thoughts of academic researchers. The MLR guidelines proposed by Garousi et al. [35] are followed to conduct this study.

Literature search process: To extract the literature from digital repositories, we developed a search string (TABLE 1). Using the strategies of quasi-gold-standard (QGS) [36] and Zhang et al. [37], we identified the keywords from five studies given in Appendix-A [PS1-PS5] against the search string element, i.e., population, intervention, and outcome [36], [38].

Population: DevSecOps. ***Intervention***: CAMS principles. ***Relevant outcome***: DevSecOps guidelines. ***Experimental***: Used research approaches.

We further identified the alternatives of keywords and used Boolean operators to concatenate them and develop the search string (see Table 1). Furthermore, in the MLR study, selecting the appropriate digital libraries for executing the search strings is important. However, we used the existing studies [36], [37], [38], [39], [40] recommendations to select the most relevant seven digital repositories (Figure 1) for exploring the pre-reviewed primary studies.

Furthermore, eight online sources were explored to collect the grey literature studies based on the MLR guidelines [35] and existing MLR studies [41], [42], [43] (see Figure 2).

Inclusion and exclusion criteria Inclusion and exclusion criteria were developed to select the most relevant studies. These criteria were developed using the suggestions provided by Garousi et al. [35] and other existing studies, e.g., Niazi et al. [44], Khan et al. [45], and Khan et al. [46].

Formal literature: All the extracted literature was initially filtered using the inclusion criteria, which helped to discover literature that fit the scope of the study: (i) The paper's full text is available, (ii) Accepted for publication in peer-reviewed journals or conference proceedings, (iii) The study provides the detailed information about DevSecOps, and (iv) The research investigates all aspects of DevSecOps. For exclusion, we used the following protocols: (i) The study does not significantly contribute to the body of knowledge, (2) the study does not give the detailed evidence of study results, (3) The study only summarizes previous research without giving any major analysis or synthesis, and (4) The study does not in the English language.

Grey literature: For performing the initial screening of grey literature, we develop the following inclusion criteria: (1) The literature is publicly available on the internet, (2) The literature is a stand-alone work written under a real name or an organization's name, (3) The content of the publication is unique, and it is longer than 250 words, and (4) the literature covers any aspect of DevSecOps. For excluding the grey literature, we used the following exclusion criteria: (1) The literature doesn't provide the details of the DevSecOps process, (2) Studies that are not in the English language, and (3) The data have not corroborated scientific outputs with real-world practices.

Quality Assessment (QA)

Formal literature: Quality of the selected studies was assessed to analyze their appropriability and significance with respect to RQs. We developed separate protocols for both types of literature to evaluate the studies' quality. The QA criteria for formal literature were developed using the systematic literature review (SLR) guidelines set by Kitchenham and Charters [47]. The quality assessment checklist is provided in TABLE 2. If a selected study explicitly covered the checklist questions, it will get 1 point score for each question. Similarly, we assigned 0.5 points if a checklist question is partially considered and gave no point if the criteria were missed entirely. The QA findings are provided in Appendix- A.

Grey literature: The quality of the grey literature material was assessed to check its significance with respect to the study objectives and RQs. We used the criteria defined by Garousi et al. [35] to perform the assessment of studies quality (see TABLE 3). The quality of grey literature material is assessed again the QA questions defined in TABLE 3. Corresponding to SLR QA scoring, we assigned 1 point if the grey literature study covered the criteria questions thoroughly. A study gets 0.5 points if the QA checklist questions are partially considered and sets 0 points for missing the QA criteria. The links of grey literature material and their respective QA score are provided in Appendix-A.

Literature selection process

Formal literature: The selected digital repositories were mined to explore relevant primary studies. Firstly, the QGS [36] was used, and five studies were selected. Secondly, an automated search process was performed by running the search string across the digital repositories and considered a total of 2055 studies after applying the initial inclusion and exclusion criteria. We applied the tollgate approach to finalize the list of selected primary studies literature [40]. The tollgate approach returned a total of 55 studies to consider for final data extraction and analysis. Furthermore, the forward and backward snowballing [42] was performed using the reference list of the selected studies. Using the snowballing approach, we collected 51 studies and refined them using the five steps tollgate approach. The tollgate screening returned 23 studies and excluded the remaining 28 primary studies. Finally, $5+55+23 = 83$ primary studies (formal literature) were selected for the final data extraction process (Appendix-A). The detail of studies selection process is given in Figure 1.

Grey literature: The grey literature was collected from 8 digital repositories as presented in Figure 2. The developed search string was executed on the selected repositories and, after applying the inclusion and exclusion criteria given in the above section, initially, 147 pieces of literature were collected. For the selection of core pieces of literature concerning the RQs of study, we apply the tollgate approach proposed by Afzal et al. [40]. After applying the five steps of the tollgate approach, 75 pieces of literature were selected for the data extraction process. The selected list of grey literature is given in Appendix-A.

Data extraction and synthesis

After finalizing the literature collection and selection process, we started data extraction from the selected studies.

TABLE 1. Keywords and their alternatives.

Search elements (SE)	Keywords
SE1 (Outcomes)	"guidelines" OR "practices" OR "motivators" OR "enablers" OR "activities" OR "techniques" OR "tools" OR "methods" OR "process."
SE2 (Intervention)	"culture", "automation", "measurement", "sharing".
SE3 (Population)	"DevSecOps" OR "SecDevOps" OR "DevOpsSec" OR "SecOps" OR ("secure AND DevOps") OR ("secure AND continuous software engineering") OR ("secure AND continuous delivery") OR ("secure AND continuous deployment") OR ("secure AND continuous integration").
SE4 (Experimental)	("grounded theory", "interviews" "case studies", "questionnaire survey", "theoretical studies", "content analyses", "action research").
Complete search string= (SE1) AND (SE2) AND (SE3) AND (SE4).	

TABLE 2. QA checklist for formal literature.

	Checklist Questions
Q1	"Does the adopted research approach answer the RQs?"
Q2	"Does the study discuss the DevSecOps guidelines?"
Q3	"Does the study provide a detailed description of the DevSecOps process?"
Q4	"Do the study findings based on empirical data?"
Q5	"Does the study explicitly focus on DevSecOps practices?"

TABLE 3. Quality assessment criteria for grey literature.

Criteria type	Questions
Authority of the producer	Q1: "Does the author have expertise in the area? (like job designation)" Q2: "Is an individual author associated with a reputable organization?" Q3: "Is the publishing organization reputable?"
Methodology	Q4: "Does the source have a clearly stated aim?" Q5: "Does the source have a stated methodology?" Q6: "Does the work cover a specific question?"
Objectivity	Q7: "Does the work seem to be balanced in the presentation?" Q8: "Does the data support the conclusions?"
Date	Q9: "Does the selected material have a clearly posted date?"
Novelty	Q10: "Does it enrich or add something unique to the research?"

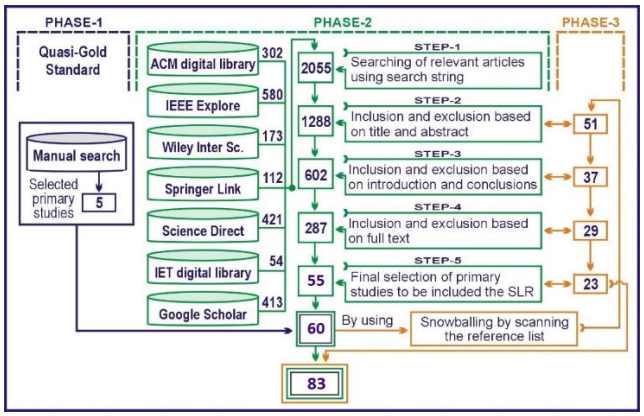

FIGURE 1. Tollgate approach for formal literature selection.

To perform the data collection from the selected literature, we established two teams and each team consists of two members. Team-1 consists of authors number 1 and 3. Team-2 includes author numbers 4 and 5. The data extraction process was performed in two phases. In the first phase, Team-1 reviewed the selected formal literature and extracted the data (i.e., ideas, themes, statements, etc.); and Team-2 extracted the data from grey literature. Author-2 (project supervisor) arbitrarily participated in both teams and validate the data extraction process. The first round of the data extraction process is completed in one month and fifteen days. During this period, both teams reviewed all selected literature. Similarly, we performed the second round of data extraction, and the selected formal literature was assigned to Team-2, and grey literature was assigned to Team-1. The second round of data extraction was completed in six weeks. Then, both teams summarized their extracted data into compact statements i.e. DevSecOps guidelines using the coding scheme [48].

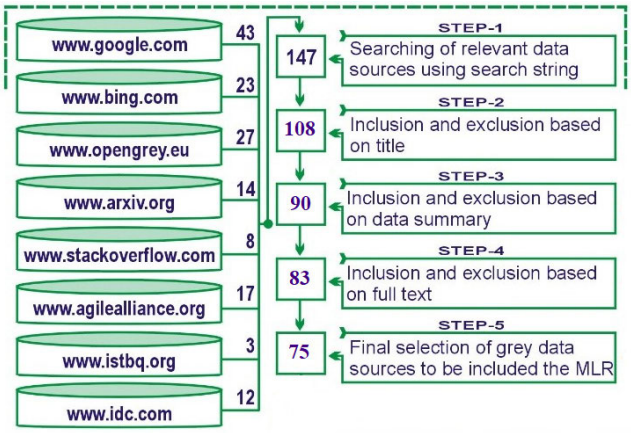

FIGURE 2. Tollgate approach grey literature section.

Once the data extraction process is finished, we performed the interrater-reliability test, to determine the baseness of the researchers' data extraction process. To perform the interrater-reliability test, three external experts were invited. They selected 15 studies and perform all the steps of the data extraction process. Considering the findings of both research teams (study authors and external experts), ''Kendall's coefficient of concordance (W) [49] was calculated. $W = 1$ and $W = 0$ present the complete agreements and disagreements between the findings of both external experts and study authors, respectively, between the findings of both data extraction teams. The results show that ($W = 0.94$) there is no baseness among the extracted data of both teams. The

TABLE 4. Kendall's coefficient of concordance test.

Data Set	Kendall Chi-Squared	df	Subjects	Raters	*p-value*	W
DevSecOps	33.35	14	15	2	0.004267	0.946765

used code is presented below, and detailed results are given in TABLE 4.

Reporting the review

Quality assessment: The QA for both data sets (grey and formal) was conducted to check the suitability of the selected studies to answer the RQs. The accumulated QA results for 81% primary studies scored ≥80% and scored ≥70% for 60% of the grey literature material. The results show the appropriateness of the selected studies to answer answering the proposed answers.

```
library(DescTools)
DevSecOps <- data.frame
(team_a=(3,3,3,4,3,4,2,3,3,2,3,3,4,5,3),
team_b=(3,4,3,4,3,4,3,4,3,3, 4, 2, 3, 3),
)
KendallW(DevSecOps, TRUE)
KendallW(DevSecOps, TRUE, test=TRUE)
)
KendallW(t(d.att[, -1]), test = TRUE)
friedman.test(y=as.matrix(d.att[,-1]), groups = d.att$id)
```

Data growth analysis: During the data extraction process, the publication years of the selected primary studies and grey literature material were also extracted to understand year-wise trends of DevSecOps studies as shown in Figure 3. The selected primary studies were published from 2015 to 2022. As the majority of the literature is published in 2020 and 2022, this shows that DevSecOps is an important research topic in the mainstream software engineering research bodies. We also noticed that the selected grey literature material was published from 2014 to 2022 with an increasing trend (see Figure 3).

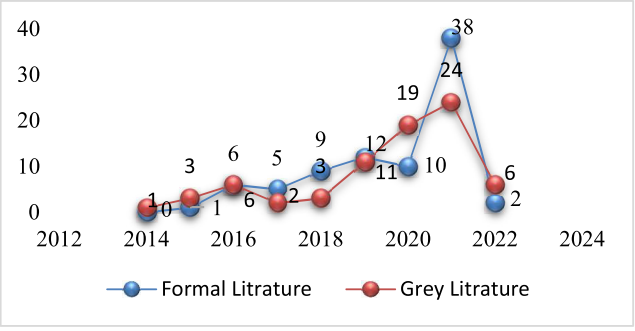

FIGURE 3. Comparison of data growth based on time.

List of identified enablers and their mapping in CAMS.

By carefully reviewing the selected primary studies and grey literature material, we first reported the list of concepts, enablers, statements, and ideas as reported in the MLR. The extracted data were firstly reviewed by the first and second authors of this study. The second round is performed by the last two authors and formats the compact statements of enablers. The final list of enablers was further mapped against CAMS (culture, automation, measurement, and sharing) principles [31], [50]. The mapping was conducted by the first two authors of this study and two industry experts from a Fin-Tec software development organization. The list of identified enablers against each CAMS principle is given in TABLE 5.

B. PROPOSED HYPOTHETICAL FRAMEWORK

This study is conducted to empirically identify the enablers that are important to potentially strengthen the CAMS principles for sustainably deploying the DevSecOps practices. Based on the literature findings (Section A), we developed the hypothetical framework of the enablers and their relationship with the CAMS principles and DevSecOps (Figure 4).

Culture: Adapting a DevSecOps culture focus on secure and high-quality software development and maintenance activities. To deliver better and secure software, it is important that the DevSecOps teams explicitly understand the security requirements of the client [14]. DevSecOps culture encourages continuous learning, experimentation, a product-oriented attitude, and a quality-oriented focus [14].

After reviewing the literature, we identified a list of 13 enablers that are significant to consider when developing a DevSecOps culture: E.1 (Building and testing of artifacts in virtual environments), E.2 (Ad-Hoc Security trainings for software developers), E.3 (Creation of threat modeling processes and standards), E.4 (Conduction of collaborative security checks with developers and system administrators), E.5 (Adopt a security champion model and implement a security requirement gathering tool), E.6 (Develop a security-oriented culture with strong security awareness), E.7 (Create a culture with a sense of shared responsibilities, where development, operations and security work together for a common goal), E.8 (Apply operational discipline to automation scripts and infrastructure security posture), E.9 (Emphasize on team empowered to make decisions), E.10 (Treat security vulnerabilities as software defects), E.11 (Prioritize fixing of security defects), E.12 (HRM programs to address challenges in culture change) and E.13 (Forming multidisciplinary teams). Therefore, the following hypothesis is formulated:

Hypothesis-1 (H1): If the identified enablers (E.1-E.13) from the literature are addressed then the development of a culture among Development, Security, and Operations teams could have a positive impact on the long-term execution of DevSecOps and makes it sustainable.

Automation: Routine tasks should be automated, resulting in a shift in which members of the software delivery team will primarily execute jobs with high variability and analyzability. In other words, automation means engineering the tasks in such a way that considers security parameters at every aspect of development and operations.

To apply automation, an organization needs to develop an organically secure development and operation structure

TABLE 5. Enablers against each CAMS principle.

CAMS	ID	Enablers	Appendix-A
Culture	E.1	Building and testing of artifacts in virtual environments	PS1, PS13, PS28, PS62, PS5, G2
	E.2	Ad-Hoc Security training for software developers	PS3, PS4, PS11, PS23, PS6, PS22, G3
	E.3	Creation of threat modeling processes and standards	PS2, PS6, PS9, PS29, PS39, G5, G11
	E.4	Conduction of collaborative security checks with developers and system administrators	PS11, PS17, PS21, PS60, G1, G4
	E.5	Adopt a security champion model and implement a security requirement gathering tool	PS19, PS27, PS28, PS46, PS59, G19
	E.6	Develop a security-oriented culture with strong security awareness	PS20, PS31, PS65, PS81, G13, G16
	E.7	Create a culture with a sense of shared responsibilities, where Dev, Ops, and security Work together for a common goal	PS40, PS52, PS54, G21, G23
	E.8	Apply operational discipline to automation scripts and infrastructure security posture	PS12, PS14, PS66, PS68, G29, G47
	E.9	Emphasize on team empowered to make decisions	PS9, PS57, PS58, PS59, G9
	E.10	Treat security vulnerabilities as software defects	PS5, PS6, PS51, G37, G60
	E.11	Prioritize fixing of security defects	PS52, PS53, G57
	E.12	HRM programs to address challenges in culture change	PS57, PS60, PS65, G61, G66
	E.13	Forming multidisciplinary teams	PS37, G62, G67, G68
Automation	E.14	Automate security as much as possible but make sure the chosen tools apply to the environment and the technology	PS23, PS25, PS44, G40, G44, G50
	E.15	Triggering out-of-band activities based on events in an automated pipeline	PS24, PS29, PS82, G31, G35, G69
	E.16	Pursue scalable governance (automate governance activities, where possible)	PS33, PS45, PS70, G28, G30, G71
	E.17	Tools for continuous vulnerability assessment	PS39, PS40, G75
	E.18	Environment depending configuration parameters	PS8, PS9, PS61, G50, G75
	E.19	Stored secrets history in git history	PS11, PS26, PS34, PS55, G11, G25, G34, G 71
	E.20	Acknowledge that automation enhances manual security activities, it does not replace them	PS62, PS73, PS82, PS83, G67, G72
	E.21	Automated process documentation	PS34, PS41, PS49, PS55, G37, G46, G74
	E.22	Framework for holistic integrity protection in microservices-based systems	PS58, PS59, PS67 PS69, PS64, G49, G70, G72
Measurement	E.23	Develop security metrics	PS38, PS40, PS64, PS68, PS69, G51, G54, G57
	E.24	Measure and monitor security constantly	PS16, PS19, PS67, PS69, G6, G39, G66, G69
	E.25	Consider the failures as an opportunity and find the root cause	PS50, PS52, PS56, PS66, PS69, G55, G62, G64
	E.26	Breaking Down Barriers and Silos with Security Champions	PS44 PS47 PS56 PS77, G51, G59, G67
	E.27	Provide a feedback loop to developers and operations on security issues and intelligence	PS34, PS37, PS38, PS49, PS69, G51, G67
	E.28	Use microservices architecture for the testing process	PS26, PS36, PS45, PS75, PS76, PS79, G37, G40, G46
	E.29	Documentation with security support	PS16, PS18, PS27, PS32, PS43, PS72, G22, G25, G28
	E.30	Clear separation of duties	PS20, PS30, PS40, PS58, PS74, G30, G33, G43
	E.31	Short feedback cycles with other teams	PS15, PS25, PS41, PS48, G12, G17, G23
Sharing	E.32	Share knowledge on security principles	PS14, PS24, PS78, G23, G36
	E.33	Coverage of service-to-service communication	PS10, PS12, PS33, PS80, G10, G19, G20
	E.34	Use dynamic access provisioning for developers in DevSecOps	PS4, PS5, PS19, PS42, PS63, G19, G40, G49
	E.35	Emphasize security concerns across all teams - not just the 'Security Team'	PS31, PS70, PS71, PS73, G12, G32, G49

and engineering teams must be mainly independent, multidisciplinary, and self-organized [13]. To add value to the table, organizations must employ automation for DevSecOps, especially for the processes that are repeatable. Automation improves DevSecOps activities by reducing the cost, effort and enhancing delivery processes [51]. It produces predictable and standardized results [51]. We explored the selected literature studies and identified the following 8 enablers which could improve the automation aspect: E.14 (Automate security as much as possible but make sure the chosen tools apply to the environment and the technology), E.15 (Triggering out-of-band activities based on events in an

automated pipeline), E.16 (Pursue scalable governance (automate governance activities, where possible)), E.17 (Tools for continuous vulnerability assessment), E.18 (Environment depending configuration parameters), E.19 (Stored secrets history in git history), E.20 (Acknowledge that automation enhances manual security activities, it does not replace them), E.21 (Automated process documentation), and E.22 (Framework for holistic integrity protection in microservices-based systems). The following hypothesis has been formulated:

Hypothesis-2 (H2): If the identified enablers (E.14-E.22) from the literature are addressing then automation could significantly impact on the long-term execution of DevSecOps and makes it sustainable.

Measurement: Continuously providing value and improvement requires systematic measurement. The measurement process can be improved by keeping track of key parameters and providing feedback. Measurement and monitoring must be considered across DevSecOps activities [33]. Callanan and Spillane [52] highlight the importance of bidirectional feedback loops for obtaining feedback from the development and operations teams. By exploring the multivocal literature, we identified the E.23 (Develop security metrics), E.24 (Measure and monitor security constantly), E.25 (Consider the failures as an opportunity and find the root cause), E.26 (Breaking Down Barriers and Silos with Security Champions), E.27 (Provide a feedback loop to developers and operations on security issues and intelligence), E.28 (Use microservices architecture for the testing process), E.29 (Documentation with security support), E.30 (Clear separation of duties) and E.31 (Short feedback cycles with other teams). Considering the above mentioned, we proposed the following hypothesis:

Hypothesis-3 (H3): If the identified enablers (E.23-E.31) from the literature are addressed then measurement could be relevant in the long-term execution of DevSecOps and makes it sustainable.

Sharing: DevSecOps is a collaboration of development, operations, and security activities. Collaboration is more focused on knowledge sharing. Every team in an organization experiences failures in terms of people, procedures, tools, projects, and so on [31]. To support successful DevSecOps deployment, this knowledge should be shared across DeevSecOps teams [31]. To make the sharing process effective in DevSecOps, literature reports the 6 important enablers i.e., E32 (Share knowledge on security principles), E.33 (Coverage of service-to-service communication), E.34 (Use dynamic access provisioning for developers in DevSecOps) and E.35 (Emphasize security concerns across all teams - not just the 'Security Team'). For this principle, our proposed hypothesis is as follows:

Hypothesis- 4 (H4): If the identified enablers (E.32-E.35) from the literature are addressed then sharing could have a positive influence in the long-term execution of DevSecOps and makes it sustainable.

According to the available academic and grey literature (experts reports, videos, white papers, blogs, and web pages). CAMS principles cover important aspects of the DevSecOps process. Thus, considering the significance of CAMS as a core pillar for the success and progress of DevSecOps, this study is planned to empirically verify the proposed hypothesis with real-world software practitioners. The empirical study indicates the practicality of the identified enablers towards the effective implementation of CAMS principles for the DevSecOps paradigm. The empirical study shows the degree of strength of identified enablers for successful consideration of the CAMS principle in the context of DevSecOps. The second research question is addressed by analyzing our findings.

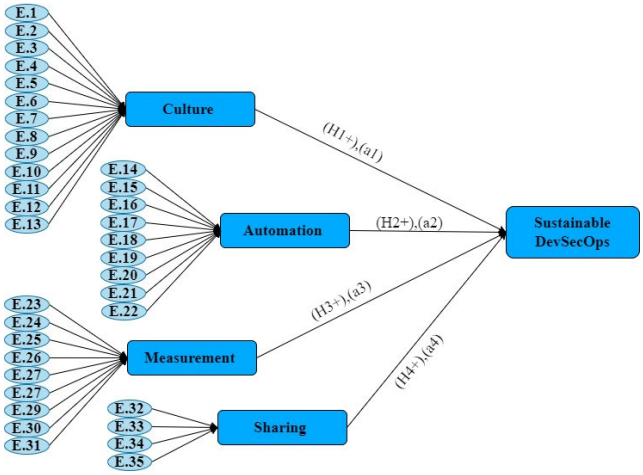

FIGURE 4. Proposed research model.

The identified enablers are independent variables for the CAMS principles (culture, automation, measurement, sharing). Similarly, CAMS principles are independent for the DevSecOps practices (See Figure 4). The regression equation for the proposed hypothetical research model is given in E.q.1.

$$\textit{DevSecOps sustainability}\ (y) = \alpha_0 + \alpha_1 v_1 + \alpha_2 v_2 + \alpha_3 v_3 + \alpha_4 v_4 \quad (1)$$

where, α_0, α_1, α_2, α_3, and α_4 are coefficients, and v_1 to v_4 are the four independent variables.

Empirical Data Collection

Kitchenham and Pfleeger [53] mentioned that it is important to consider the right data collection approach focusing on the type of data, "resources", "execution techniques of selected approach" and "skill to operate the variable of interest". While collecting the observational types of data, it is hard to collect a solid and representative set of data [54], [55]. Questionnaire-survey is one of the powerful approaches to get feedback from geographically distributed and most potential populations related to the study objectives. Therefore, to get the practitioner's insight, we used a questionnaire survey approach. The survey instrument was developed in three different sections (available on Appendix-B). In the first section, questions deal with participants' demographic information. The second section contains the questions regarding participants' organization. The third section is further categorized as (Section III-A) which contains the questions regarding independent variables which were developed using the identified

enablers against CAMS principles. Section III-B contains the variables against dependent variables.

We used the five-point Likert scale ranging from 1 to 5, whereas 1 presents the strongly disagree and 5 presents the strongly agree. The questionnaire also has open-ended questions which allow the survey participants to report additional enablers that are not identified during the literature survey (See Section III-A).

A pilot assessment of the questionnaire was performed to improve the understandability and readability [56], [57], [58]. The questionnaire was shared with three industry experts and two academic researchers working in DevSecOps domain. The experts were requested to fill the questionnaire and give their suggestions. The suggestions were mainly related to the questionnaire structure and readability of the questions. The questionnaire was finally updated based on the expert's suggestions.

For the collection of appropriate data concerning the research objective, it is important to target the right population (i.e., practitioners have experience of dealing with security in DevOps or working with DevSecOps). The targeted population was approached using various social media networks (Facebook and LinkedIn) and further used a snowballing technique to spread the questionnaire among the targeted community [59], [60]. Secondly, we visited 11 software development organizations working with DevOps and Secure DevOps, in Finland and the United Arab Emirates. The data collection lasted two and half months and we approached around 765 practitioners. However, we gathered 160 responses from social media links and got complete and usable responses from 143 practitioners. From the 73 in-person visits, we got 71 valid responses (Finland practitoners 19, and United Arab Emirates 52). It means that all the responses were carefully reviewed, and the collected data were cleaned by removing 19 incomplete responses. Therefore, we included 214 responses for the final data collection.

Survey data analysis

We used the Structural Equation Modeling (SEM) approach of Partial Least Square (PLS). PLS-SEM is widely used for making predictions and for developing theories when working with complex models [61], [62], [63]. Smart-PLS was chosen in this study due to its robustness in handling complex models with multiple latent variables and its suitability for exploratory research with moderate sample sizes. Unlike covariance-based structural equation modeling, Smart-PLS does not require a large sample size or strict data normality, making it ideal for this study's dataset."

The flexibility of Smart-PLS in assessing reflective and formative measurement models, coupled with its ability to simultaneously evaluate measurement and structural models, ensures accurate and reliable results for validating the proposed hypotheses.

We evaluated the proposed hypotheses (H1 to H4), using PLS-SEM using three different steps. In step 1, we performed the normal distribution and parametric statistical tests. In this step, a parametric statistical method was used, and the Pearson correlation of coefficient was determined for tests with a one-tailed t-test for the proposed hypothesis i.e., H1 to H4.

The PLS is a more effective approach for analyzing the data compared with traditional techniques e.g., regression. PLS enables the researchers to simultaneously model the relationship between multiple independent and dependent variables and respond to the research questions in a single, systematic, and comprehensive analysis [88]. In addition, PLS is appropriate for this research work as it gives the structural model that is hard to analyze relationships among a large set of variables [64], [65]. It is worth to note that various recently conducted studies in the domain of software engineering use the same technique [66], [67], [68], [69].

The data analysis process followed two main steps: measurement model evaluation and structural model evaluation. The measurement model evaluation assessed internal consistency reliability (using Cronbach's Alpha and Composite Reliability), convergent validity (using Average Variance Extracted (AVE)), and discriminant validity (using the Fornell-Larcker Criterion). The structural model evaluation included path coefficient estimation, hypothesis testing using bootstrapping (5,000 resamples), and assessing the explanatory power of the model using R^2 values. Additionally, model fitness was evaluated using Standardized Root Mean Square Residual (SRMR), Normed Fit Index (NFI), and Rms_Theta metrics.

IV. RESULTS

According to Chin [70], the results of PLS should be determined in two different phases including measurement model evaluation and structural model evaluation. In structural model analysis, significance structure path, coefficient of determination (R2), and estimations of path coefficients were employed, and the results are presented in the following sections.

A. MEASUREMENT MODEL EVALUATION

1) DEMOGRAPHIC PROFILE

The demographic information shows that most of the survey participants were either project managers or software quality managers. These results are presented in Figure 5. We further analyzed the demographic data to check the organization size of survey participants. The results presented in Figure 5 reveal that 91 (66%) respondents belong to small organizations, 49 (36%) are medium size organizations, and 74 (54%) are from large-scale. The results show that the survey responses are equally distributed across the organization size. Therefore, it seems that the results of this study could be generalized regardless of the organizational size.

Figure 5 also shows the experience level of survey respondents ranging from 2 to 20 years. The mean and medium were calculated, and the results (6 and 5.5 respectively) indicate a young pool and experienced pool of respondents. As a result,

TABLE 6. Reliability statistics.

Variables	Cronbach's Alpha	Composite Reliability	Original No. of Items	Final No. of Items
Culture	0.909	0.933	13	13
Automation	0.954	0.961	9	9
Measurement	0.899	0.922	9	9
Sharing	0.901	0.922	4	4

there is a good mix of survey participants with varying levels of experience in software development activities. Finally, we noticed that males are dominating in the software development field as we observed that out of 214 survey respondents 45 (21%) were female and 169 (79%) were male. Out of 214 respondents, the highest frequency of participants is from Finland (43, 20%), followed by UAE (34, 16%), India (27, 13%), and China (21, 10%).

2) DATA NORMALITY

To measure the normality of the collected data, we applied the Skewness and Kurtosis test. Based on the results of skewness and kurtosis, there is an approximately normal distribution of the collected data. Concerning George and Mallery [71], values ranging from 1 to -1 and 2 to -2, present the normal and satisfactory normal data, respectively.

3) INTERNAL CONSISTENCY

For internal consistency, the common measures are Cronbach's Alpha and Composite reliability.

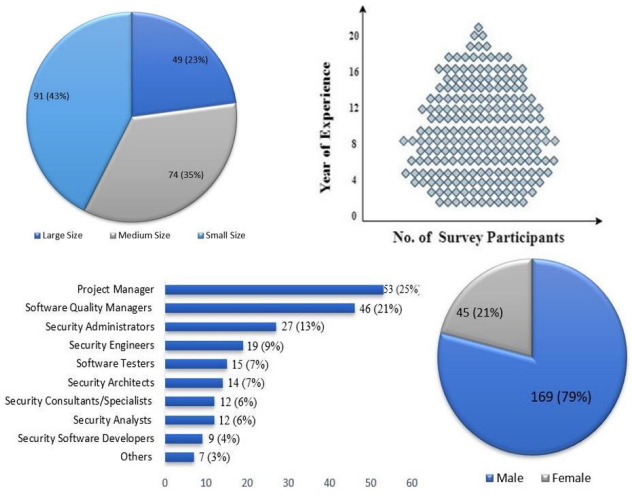

FIGURE 5. Demographics of survey participants.

According to the results of Cronbach's Alpha and Composite reliability presented in TABLE 6, all the variables of this study are satisfactory. Considering the definition of Henseler et al. [72], a value of 0.70 or above is considered as satisfactory internal consistency. To come up with internal consistency, all the variables were considered on a personality traits scale.

4) INDICATOR RELIABILITY

To measure the indicator reliability, we used the outer loading scheme, and the results are presented in Table 7. The results of the measurement model showed satisfactory reliability, with Cronbach's Alpha and Composite Reliability values exceeding the recommended threshold of 0.7 for all constructs. Convergent validity was confirmed as the AVE values were above 0.5 for all variables, and discriminant validity was established through the Fornell-Larcker Criterion."

The model fitness was confirmed through SRMR (0.069), NFI (0.567), and Rms_Theta (0.199), all of which were within acceptable thresholds, indicating a well-fitting structural model.

The results of outer loading are considered satisfactory if the value is 0.70≥ [72]. According to the results presented in TABLE 7, all the variables are satisfied according to the indicator reliability testing.

5) MULTICOLLINEARITY

The multiple regression equation presents the multicollinearity among two or more variables, and it is measured using Variance Inflation Factor (VIF) values. If a VIA value is 10≥, it indicates a multicollinearity issue [73]. By considering the determined VIF results given in TABLE 8, there is no multicollinearity issue.

6) VALIDITY ANALYSIS

To measure data validity, we used convergent and discriminant validity analysis. The convergent validity was measured using Average Variance Extracted (AVE) values. If the determined value of AVE is above 0.5, it is considered satisfactory [72]. According to the results presented in TABLE 9, all the variables attain the satisfactory value for convergent validity.

Secondly, the discrimination validity was measured. One of the criteria used to determine PLS is the Forner-Lorcker criterion. The determined results of the Forner-Lorcker criterion for discrimination validity are given in Table 10. Forner and Larcker [74] underlined that if the square root AVE for each variable is greater than the correlation between the variables, it indicates that the data set attained acceptable discriminant validity [72], [75].

According to the results given in TABLE 10, all the variables fulfill the satisfactory discrimination validity value. The diagonal value presents the square roots of the corresponding variables, and the value below the square root presents the correlation between variables. Considering Layman's terms, the diagonal values should be greater than the values below the diagonal values.

7) MODEL FITNESS

The PLS offers different measures for model fitness e.g., SRMR, NFI, and rms_Theta. The determined results for these models are presented in TABLE 11. Standard Root Mean Square (SRMR) based on bootstrap in PLS is equivalent to

TABLE 7. Indicator reliability through outer loadings.

ID	Culture	Automation	Measurement	Sharing	DevSecOps
E.1	0.816				
E.2	0.867				
E.3	0.764				
E.4	0.721				
E.5	0.741				
E.6	0.864				
E.7	0.832				
E.8	0.857				
E.9	0.836				
E.10	0.859				
E.11	0.751				
E.12	0.831				
E.13	0.857				
E.14		0.835			
E.15		0.853			
E.16		0.901			
E.17		0.869			
E.18		0.846			
E.19		0.828			
E.20		0.903			
E.21		0.846			
E.22		0.918			
E.23			0.864		
E.24			0.861		
E.25			0.918		
E.26			0.843		
E.27			0.850		
E.28			0.923		
E.29			0.846		
E.30			0.828		
E.31			0.869		
E.32				0.735	
E.33				0.716	
E.34				0.710	
E.35				0.864	
Culture					0.877
Automation					0.839
Measurement					0.864
Sharing					0.844

TABLE 8. Multicollinearity values.

Independent Variables	Sustainability Intention	Sustainability
Culture	3.776	
Automation	2.324	
Measurement	2.763	1.000
Sharing	4.746	

TABLE 9. AVE values variables.

Variable	AVE
Culture	0.732
Automation	0.769
Measurement	0.701
Sharing	0.667

chi-square in other techniques [62], [76]. In SRMR, if the above than 0.1 renders the problem in model fitness [62]. Considering the results presented in Table 11, the determined value of SRMR is below 0.1, and this shows the fitness of the proposed model. It is also noticed that the NFI is also greater

TABLE 10. Fornell-Larcker criterion for discriminant validity.

	1	2	3	4
Culture (1)	0.857	-	-	-
Automation (2)	0.695	0.864	-	-
Measurement (3)	0.847	0.783	0.837	-
Sharing (4)	0.818	0.788	0.832	0.794

TABLE 11. Model fitness.

	Saturated Model	Estimated Model
SRMR	0.069	0.087
NFI	0.567	0.563
Rms_Theta	0.199	

than 0.50 and closer to 1, and this indicated that the proposed model is a good fit. Furthermore, we noticed that the value of rms_Theta is closer to 0, and this indicates the good fitness of the model.

Structural Model Evaluation: In this section, the proposed hypotheses are tested considering their effect and significance. Bootstrapping was applied to measure the degree (estimate values), significance (P Values and T-statistics), and R^2 measure of the structural model [62]. In this test, T-statistics is significant, if the calculated value is greater than 1.96 (at significance level 5%) or 1.65 (significance level 10%). In the following sections, we analyzed each relationship considering the parameter estimates (beta values) and T- statistics. The results are presented in Figure 6 and TABLE 12, indicating the causal structural model results.

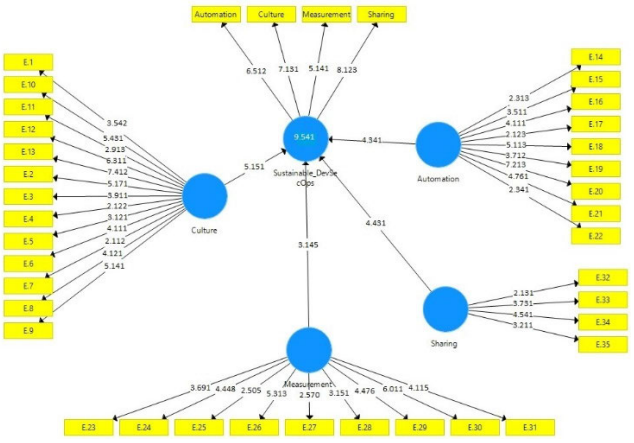

FIGURE 6. Structural model evaluation.

a: CULTURE AND DevSecOps SUSTAINABILITY

According to the results (P-Value: 0.000; T-value: 4.260) given in Table 12, the results of the culture category for sustainable DevSecOps are significant and positive considering the beta value (Beta value: 0.138). Hence, the results show that effective implementation of enablers identified against the culture category could significantly contribute to the sustainable DevSecOps adoption for the software engineering process; and this renders that the H1 is accepted.

TABLE 12. Bootstrap results for causal structural model.

	Parameter Estimate	Sample Mean	Standard Deviation	T Statistics	P Values
Culture→ DevSecOps Sustainability	0.138	0.138	0.032	4.260	0.000
Automation → DevSecOps Sustainability	0.788	0.789	0.029	27.058	0.000
Measurement → DevSecOps Sustainability	0.271	0.268	0.066	4.088	0.000
Sharing → DevSecOps Sustainability	0.487	0.487	0.051	9.538	0.000

b: AUTOMATION AND DevSecOps SUSTAINABILITY

The results presented in Table 12 (P-Value: 0.000; T-value: 27.058) reveal that there is a positive relationship between automation and sustainable DevSecOps adoption. We also noted that concerning the beta value (0.788), there is a significant positive relationship among both variables. Therefore, considering the given results, H2 is supported by the opinions collected from real-world practitioners.

c: MEASUREMENT AND DevSecOps SUSTAINABILITY

Considering the results (P-Value: 0.000; T-value: 4.088) presented in Table 12, there is a positive relationship between the third principle of CAMS model ´Measurement´and DevSecOps adopting sustainability in software development organizations. Besides this, the beta value (0.271) indicated that both measurement and DevSecOps sustainability has a strong and significant relationship. This shows that by properly measuring the DevSecOps process, the practitioners could make it more sustainable for the software engineering process. Hence, H3 is accepted, and it is assumed that measurement is important for sustainable DevSecOps execution in the software development process.

d: SHARING AND DevSecOps SUSTAINABILITY

The results indicate that there is a positive relationship between sharing and DevSecOps sustainability. Based on the results given in Table 12 (beta value = 0.487, T-Value = 9.538, P-value = 0.000), we could assume that the information-sharing environment between DevSecOps teams could significantly contribute to making the DevSecOps activities sustainable in software development organizations. Therefore, the proposed H4 is accepted.

e: COEFFICIENT OF DETERMINATION FOR THE ENDOGENOUS LATENT VARIABLES

To check the variance that carried by independent variables to the dependent variable, we determined the coefficient

TABLE 13. R2 value for endogenous latent variables.

	Original Sa	N	SD	T Stat	P Val
DevSecOps	0.795	0.800	0.025	32.624	0.000

of determination (R2), and the results are presented in TABLE 13. The analyzed results of R2 between exogenous variables which include four CAMS principles, and the endogenous variable (Sustainable DevSecOps) are substantial and significant as R2 = 0.795 and T = 32.624.

This presents that the successful implication of culture, automation, measurement, and sharing, together, amount to a significant (79.5%) for DevSecOps sustainable execution in a software development environment. Senapathi and Srinivasan [66] mentioned that if the R2 $\geq$ 0.75 is considered substantial.

V. DISCUSSION

The objective of this study is to understand the enablers that could assist in making DevSecOps adoption sustainable in software development organizations. To come up with the objective of this study, we have studied the following research questions:

[RQ1] What enablers of sustainable DevSecOps are reported in the literature related to CAMS principles?

To answer this, we conducted an MLR aiming to investigate the DevSecOps enablers against each principle of CAMS. We have collected most potential literature published as scientific research and grey literature, including experience reports, blogs, white papers, case studies, etc. By applying the MLR protocols, we have selected 83 pieces of formal data and 75 pieces of grey literature for data extraction. All the selected pieces of literature were reviewed and identified 35 DevSecOps enablers against CAMS principles.

[RQ2] What are the practitioner's perceptions regarding the literature-based enablers of sustainable DevSecOps?

Based on the literature findings, we developed the research hypothesis and proposed a hypothetical framework. Furthermore, we conducted a questionnaire survey study aiming to check experts' opinions with respect to the proposed hypothetical framework and the results are discussed in the subsequent sections:

A. CULTURE

Hypothesis-1 (H1): If the identified enablers (E.1-E.13) from the literature are addressed then the development of a culture among Development, Security, and Operations teams could have a positive impact on the long-term execution of DevSecOps and makes it sustainable

According to the results, there is a positive correlation between culture and DevSecOps sustainability (P-Value: 0.000; T-value: 4.260). The analyzed results predict that culture could significantly contribute to make the DevSecOps process activities sustainable in a software development environment. DevSecOps culture focuses on bringing together the traditionally independent silos of development, security, and operations into a collaborative shared-responsibility model. The identified list of enablers (see Table 5), gives a good insight to make the culture in line with the DevSecOps paradigm requirement.

Casey [77] highlighted the importance of DevSecOps culture and defined that organizations should consider security as a culture. They went on to say that DevSecOps culture is well-suited to hybrid computing environments, faster and more frequent software delivery, and other current IT demands. That is one of the reasons why DevSecOps is important to IT executives. It is also the most difficult part: Culture change makes something as simple as updating an out-of-date tool appear simple [77].

Security and DevOps teams mostly do not work under the same umbrella, because they often have widely diverse goals. One focused on features and functionality while the other on mitigating security risk. This lack of cohesion between teams is a detriment to organizations as well as the people both groups seek to serve [78]. How can security and DevOps teams work together to arrive at a healthy DevSecOps culture, one where all three areas (Dev, Sec, and Ops) are in tune. Furthermore, Chiodi [78] suggested that the first step to making the DevSecOps healthy is the understanding of the existing culture in which the teams work.

B. AUTOMATION

Hypothesis-2 (H2): If the identified enablers (E.14-E.22) from the literature are addressing then automation could significantly impact on the long-term execution of DevSecOps and makes it sustainable.

The results of PLS show that (P-Value: 0.000; T-value: 27.058), there is a positive relationship between automation and sustainable DevSecOps adoption. Thus, to make the automation effective, DevSecOps adopters need to consider the enablers explored and reported in section III-A and Table 5.

The manual running of security tools across the entire application source code each day can be time-consuming and hinder your ability to keep up with daily changes [31]. For security checks to keep pace with code delivery in a CI/CD environment, security automation is important. Security products need to integrate into the development pipeline and enable both the development and security teams to work together instead of just throwing things over the fence at each other [51].

The advantages of DevSecOps automation seem obvious: streamlined, collaborative development, security, and operations [12]. The practice of DevSecOps is primarily reliant on automation to produce in a matter of hours and regularly across platforms. As a result, DevSecOps automation promotes speed, accuracy, consistency, and reliability while also increasing the number of delivery. Automation also plays a key role in processes designed to find and highlight security

vulnerabilities in code. The clarity in code offered by DevOps and DevSecOps pipelines also makes it clear who is best suited to solve specific issues [51].

By applying effective automation, issues are more likely to be found and repaired earlier on, so much so that more problems can be solved within the timeframe before release, resulting in higher-quality end products. DevSecOps engineers are also equipped with knowledge of the best security tools and practices to keep everything running as smoothly as possible [31], [79].

With the speed and reliability offered by automating DevSecOps, practitioners can often find themselves with more free time for other tasks. This can be applied to solving problems that the organization may not previously have had the time and resources to deal with [80]. By integrating security and compliance checks into a DevOps pipeline, DevSecOps engineers make compliance a matter of utmost importance. Checks are added throughout the pipeline, with automated traceability ensuring that both problems and their causes can be found and fixed as soon as possible [80].

C. MEASUREMENT

Hypothesis-3 (H3): If the identified enablers (E.23-E.31) from the literature are addressed then measurement could be relevant in the long-term execution of DevSecOps and makes it sustainable.

The PLS results (P-Value: 0.000; T-value: 4.088) indicated that measurement is an important aspect to develop sustainable DevSecOps. Furthermore, the results also show that the enablers mapped against the measurement category (see Table 5) are important to make the measurement process appropriate in the DevSecOps process.

In the current era of end-to-end DevSecOps, firms need to evaluate the key performance points to prove a success and drive further transformation. DevSecOps is a journey that requires continuous refinement. Besides, if organizations are not evaluating the consequences regularly, it will be very difficult to expand DevSecOps execution in the complete environment [31]. Morales et al. [81] emphasized the importance of DevSecOps measurement given its significance to determine how practitioners work. The measure is important to assess DevSecOps results. Evaluation through DevSecOps is essential to make the process efficient and to get effective outcomes [14]. Because measurement is so influential when done well, it is the secret ingredient in any DevSecOps transformation. It is an important part of any technology transformation: with metrics in hand, you will be able to take use of the insights you receive from your trials, capitalize on your achievements, and pivot swiftly if anything you have built is not working [82]. Small improvements lead to big benefits, and with metrics in hand, organizations be ready to capitalize on them.

D. SHARING

Hypothesis- 4 (H4): If the identified enablers (E.32-E.35) from the literature are addressed then sharing could have a positive influence in the long-term execution of DevSecOps and makes it sustainable.

The knowledge-sharing is essential not only for newly onboarded people to grow but also for the software development process to improve. Developing software requires the collaboration of multiple people who are experts in various fields and are all impacted by one another's performance. These individuals must share their knowledge to deliver high-quality products [31]. The result of this empirical study (T-Value = 9.538, P-value = 0.000) shows that sharing is a significant pillar to make DevSecOps sustainable in software development organizations. The results also indicated that the identified enablers with respect to Sharing (see Table 5) are important to make sharing activities effective and efficient.

One of the most defining features of DevSecOps is that it breaks down silos between different teams [31]. In DevSecOps the development and operational and security teams work as a joint force to share insights, skills, and expertise while also improving each other's practices and processes. Security specialists communicate with colleagues and upskill them in security considerations and vice versa. They will also clarify elements such as who is best suited to fixing certain issues and how everyone can help meet security targets [32]. In essence, DevSecOps helps all non-security staff understand how their own goals and practices align with and are impacted by security. The approach improves their awareness and contributes to the efficiency and effectiveness of the overall pipeline [32].

VI. LIMITATIONS

Like other studies, this research also has some limitations. Firstly, the enablers were identified from the multivocal literature, and there might be a threat of omitting related literature. However, as previous systematic literature review studies have noted [44], [55], [57], [83], this threat is not a systematic omission. We clearly defined the review process (section III-A), including multiples database repositories and also applying snowballing. By consulting previous literature related to the topic, e.g., [44], [55], [57], and [83], we identified the search terms considering the RQs and understanding the objectives of this study. These search terms were further used to formulate the final search string by discussing with all the authors. Moreover, authors of this study have significant experience in conducting systematic literature review studies [14], [26], [43], [84]. Although the increasing numbers of publications on the topic might cause the missing of some relevant studies, we are confident that the results of our study are comprehensive enough to provide a good overview of the published grey and formal literature.

Another threat to the validity of MLR findings is the researchers' biasness, and that leads to continuously extract the biased data. To address this threat, we conducted an interrater-reliability test (section III-A, Table 4) [49], which reveal that there no concrete bias existed among the data extraction team.

Construct validity is another threat related to the measuring scales and whether they accurately represent the attributes being measured in a questionnaire survey. A possible threat is that the survey- participants might have a problem interpreting the variables. To address this threat, we have conducted meeting with some of the respondents physically and via Zoom. In addition, we mentioned contact details on the first page of the questionnaire with the intent to reach out to the research team. Nevertheless, we did not receive any complaints from survey participants regarding the understandability of the questionnaire. And this renders the used questionnaire survey was understandable for all the participants.

DevSecOps is a new software development paradigm and few organizations working with it, due to this, it is very difficult to approach and collect the data from people with relevant expertise. Thus, an important threat to the validity of this study is the small sample size of collected empirical data, i.e., $N = 214$. Although according to the general rules of statistics, the collected data sample is justifiable for generalization of study results [85], [86]., generalizing the findings should be done with caution because the study's respondents are from Finland and United Arab Emirates (UAE).

For empirical investigations a survey instrument was developed, and to check the appropriability and understandability of the developed questionnaire, the pilot assessment was performed. Considering the suggestions and recommendations of experts, the survey-instruments were modified, and the updated version of the survey instrument was used for data collection. This gives the acceptable internal validity of the data collection instrument.

VII. STUDY IMPLICATIONS

This research work has both research and industrial implications.

The findings of the study give an insight into state-of-the-art literature and practices related to the important enablers across CAMS principles in sustainable DevSecOps. The identified list of enablers against the core 4 principles of DevSecOps could serve as a knowledge base to develop effective and efficient tools and strategies for sustainable DevSecOps adoption. Empirical results give evidence to the research community as the identified enablers are core areas that need to be addressed in future research to make the DevSecOps process sustainable.

In addition, industry experts can use the findings of this study in several ways. The study provides a set of enablers that practitioners could adopt to foster a long-term execution of DevSecOps and makes it sustainable. Furthermore, the identified list of enablers serves as a body of knowledge for practitioners to better plan the DevSecOps activities. The organizations also consider the identified set of enablers to enhance their capabilities of DevSecOps process implementation by offering training opportunities to practitioners in targeted areas where skill development is required.

The practitioners may also find the study results helpful in focusing on the enablers identified against each core principle of DevSecOps (i.e., culture, automation, measurement, and sharing). As the identified enablers against each principle could be helpful as an indicator to hire the team members considering the specific skills as it could be a risk mitigation strategy for DevSecOps based projects.

The finding of this study is also important for organizations to understand their ability to adopt DevSecOps as a software development paradigm; as the list of enablers mapped against 'culture', 'automation', 'measurement' and 'sharing' indicate the important areas of DevSecOps.

Improvements to weak DevSecOps process can be better planned and managed rationally and targeted using insights provided by the findings of our study. Ultimately, the results of this study will help organizations to make the DevSecOps process successful and sustainable, which could lead to the development of quality software projects.

VIII. RELATED WORK

DevSecOps is a relatively new software development paradigm, and few studies have been conducted to address the problems faced by practitioners while considering security from development to delivery. For example, Mohan and Othmane [87] conducted a systematic mapping study that only included five primary studies and conference presentations. The purpose of their research was to determine whether DevSecOps or SecDevOps was just a buzzword during that period. They also identified the list of DevSecOps challenges and their possible solutions. Similarly, Myrbakken and Colomo-Palacios [12] conducted a MLR, and placed the challenges, benefits, definition, characteristics, and evolution of DevSecOps practices. They also underlined the challenges related to traditional or manual security approaches, organizational problems, and lack of appropriate tools. Furthermore, Myrbakken and Colomo-Palacios [12] reviewed 13 studies and highlighted the metrics that need to be adopted while measuring the effectiveness of DevSecOps.

Another effort conducted by Sánchez-Gordón and Colomo-Palacios [14] studies the cultural aspect of DevSecOps and mentions important areas that need to be considered by practitioners for effective DevSecOps culture. Mao et al [32] conducted a grey literature study and investigated the DevOps impact on software security from practitioners' perspectives. They recommended some solutions to understand the implacability of DevSecOps for secure software development and execution of DevSecOps practices in a real-world environment. An SLR study conducted by Rafi et al [88] highlighted identified software security challenges while using DevOps as a software development paradigm. They further prioritize the identified challenges and develop a prioritization-based taxonomy.

We noticed two empirical studies in the DevSecOps domain. Akbar et al. [26] developed a decision-making framework of DevSecOps challenging factors using a hybrid approach. Firstly, they conducted a literature review and

identified the 18 challenging areas important to address for DevSecOps success. They conducted an empirical study and developed a prioritization-based taxonomy based on experts' feedback. Tomas et al. [31] interviewed six practitioners to get their insights regarding CAMS principles in DevSecOps practices. They highlighted the importance of creating a security culture in their software products. Authors suggest necessary training sessions to educate the team members on effective security automation tools. They also recommend the application of continuous measurements to keep track of identified vulnerabilities, the amount of training staff has received, and staff's general opinions regarding security.

According to the literature, security should be considered an important aspect of DevOps. Existing studies [12], [14], [31], [87] show that little research has been conducted to understand DevSecOps as well as on the enablers that could positively impact on DevSecOps adoption. No comprehensive study has been undertaken to understand the enablers to make DevSecOps sustainable as well. Compared to these empirical studies [26], [31], our study is novel as we firstly identified the core enablers by conducting a comprehensive MLR and developed a hypothetical framework. Secondly, to check the applicability of the proposed hypothetical framework, we conducted a questionnaire survey study with real-world practitioners. Both existing empirical studies on DevSecOps [26], [31] highlight the main areas and challenging factors of DevSecOps. Considering the significance of DevSecOps sustainability in software engineering, this study explored the critical enablers against CAMS principles from state-of-the-art literature and proposed a hypothesis against each principle (section IV-A). Furthermore, the proposed hypotheses are empirically assessed by conducting a survey questionnaire with real-world practitioners.

IX. CONCLUSION AND FUTURE DIRECTIONS

The core objective of this research was to understand the concept of DevSecOps, CAMS (culture, automation, measurement, and sharing) principles, and the enablers that are important for sustainable DevSecOps practices. To achieve the mentioned objective, 35 enablers were identified from the literature survey, and an empirical study was conducted to evaluate them in the industrial domain. The empirical study participants were those who have experience working on DevSecOps or DevOps projects. The study findings show that it is important to effectively address the 4 pillars of DevSecOps (i.e., CAMS principle). Furthermore, the results also illustrate that practitioners should consider the enablers that are explored and mapped against culture, automation, measurement, and sharing for developing a sustainable DevSecOps environment. The identified enablers specify the core areas that are important for effectively managing the DevSecOps practices. We believe the findings of this study will help to understand DevSecOps principles and the key areas that need to be considered by industry experts to make DevSecOps sustainable.

In the future, we plan to extend this work to develop a comprehensive decision-making framework to assist practitioners to make DevSecOps successful and sustainable. We plan to conduct an interview study and use the grounded theory approach, to investigate the best practices, success factors, and barriers of DevSecOps. Then, we will develop a decision-making framework using the findings of interview. Furthermore, we will conduct a set of case studies with different organizations to check the usefulness of the decision-making framework in a real-world environment and will update it according to the feedback that we will receive from case studies. The final developed framework will help the software development community to successfully apply DevSecOps in their production units.

STATEMENTS AND DECLARATIONS

The authors have no competing interests to declare that are relevant to the content of this article.

APPENDIX A
SELECTED LITERATURE ALONG WITH QA SCORE

https://tinyurl.com/2pj8eyt8

APPENDIX B
QUESTIONNAIRE SURVEY

https://forms.gle/J7Avshuzn85K5UM28

ACKNOWLEDGMENT

This work was supported and funded by the Deanship of Scientific Research at Imam Mohammad Ibn Saud Islamic University (IMSIU) (grant number IMSIU-RP23060).

REFERENCES

[1] E. C. Emenike, "Integration of sustainable development in software development: Case study: Wedding planning," LUT Univ., Finland, Tech. Rep. 601, 2019.

[2] H. Winschiers-Theophilus and B. Paterson, "Sustainable software development," in *Proc. Annu. Res. Conf. South Afr. Inst. Comput. Scientists Inf. Technologists IT Res. Developing Countries*, Oct. 2004, pp. 274–278.

[3] A. Z. Ibatova, F. F. Sitdikov, and G. S. Klychova, "Reporting in the area of sustainable development with information technology application," *Manage. Sci. Lett.*, vol. 8, no. 7, pp. 785–794, 2018.

[4] R. Goodland, "Sustainability: Human, social, economic and environmental," *Encyclopedia Global Environ. Change*, vol. 5, pp. 96–174, Jan. 2002.

[5] B. Penzenstadler, "Towards a definition of sustainability in and for software engineering," in *Proc. 28th Annu. ACM Symp. Appl. Comput.*, Mar. 2013, pp. 1183–1185.

[6] D. Wiktor-Mach, "What role for culture in the age of sustainable development? UNESCO's advocacy in the 2030 agenda negotiations," *Int. J. Cultural Policy*, vol. 26, no. 3, pp. 312–327, Apr. 2020.

[7] B. Penzenstadler and H. Femmer, "A generic model for sustainability with process- and product-specific instances," in *Proc. Workshop Green Softw. Eng.*, Mar. 2013, pp. 3–8.

[8] L. Leite, C. Rocha, F. Kon, D. Milojicic, and P. Meirelles, "A survey of DevOps concepts and challenges," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–35, Nov. 2020.

[9] L. E. Lwakatare, P. Kuvaja, and M. J. I. Oivo, "An exploratory study of devops extending the dimensions of devops with practices," *Icsea*, vol. 104, p. 2016, Aug. 2016.

[10] F. M. A. Erich, C. Amrit, and M. Daneva, "A qualitative study of DevOps usage in practice," *J. Softw., Evol. Process*, vol. 29, no. 6, p. 1885, Jun. 2017.

[11] K. Carter, "Francois Raynaud on DevSecOps," *IEEE Softw.*, vol. 34, no. 5, pp. 93–96, Sep. 2017.
[12] H. Myrbakken and R. Colomo-Palacios, "DevSecOps: A multivocal literature review," in *Proc. Int. Conf. Softw. Process Improvement Capability Determination*. Cham, Switzerland: Springer, Jan. 2017, pp. 17–29.
[13] K. T. Gomes, "The importance of DevSecOps," Northern Illinois Univ., USA, Tech. Rep. 1214, 2018.
[14] M. Sánchez-Gordón and R. Colomo-Palacios, "Security as culture: A systematic literature review of DevSecOps," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. Workshops*, Jun. 2020, pp. 266–269.
[15] W. F. Santoso and D. S. S. Sahid, "Implementation and performance analysis development security operations (DevSecOps) using static analysis and security testing (SAST)," in *Proc. Proc. Int. Appl. Bus. Eng. Conf.*, Aug. 2021, pp. 17–19.
[16] A. Koskinen, "DevSecOps: Building security into the core of DevOps," M.S. thesis, Fac. Inf. Technol., Univ. Jyväskylä, Jyväskylä, Finland, 2019.
[17] B. Fitzgerald and K.-J. Stol, "Continuous software engineering: A roadmap and agenda," *J. Syst. Softw.*, vol. 123, pp. 176–189, Jan. 2017.
[18] M. Leppänen, S. Mäkinen, M. Pagels, V.-P. Eloranta, J. Itkonen, M. V. Mäntylä, and T. Männistö, "The highways and country roads to continuous deployment," *IEEE Softw.*, vol. 32, no. 2, pp. 64–72, Mar. 2015.
[19] L. Chen, "Continuous delivery: Huge benefits, but challenges too," *IEEE Softw.*, vol. 32, no. 2, pp. 50–54, Mar. 2015.
[20] M. Shahin, M. Ali Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.
[21] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Reading, MA, USA: Addison-Wesley, 2015.
[22] S. Rafi, W. Yu, M. A. Akbar, S. Mahmood, A. Alsanad, and A. Gumaei, "Readiness model for DevOps implementation in software organizations," *J. Softw., Evol. Process*, vol. 33, no. 4, p. 2323, Apr. 2021.
[23] M. A. Akbar, W. Naveed, S. Mahmood, A. A. Alsanad, A. Alsanad, A. Gumaei, and A. Mateen, "Prioritization based taxonomy of DevOps challenges using fuzzy AHP analysis," *IEEE Access*, vol. 8, pp. 202487–202507, 2020.
[24] D. Shackleford, "The devsecops approach to securing your code and your cloud," SANS Inst. InfoSec Reading Room A DevSecOps Playbook, MD, USA, Tech. Rep., 2017.
[25] A. A. U. Rahman and L. Williams, "Software security in DevOps: Synthesizing practitioners' perceptions and practices," in *Proc. IEEE/ACM Int. Workshop Continuous Softw. Evol. Del. (CSED)*, May 2016, pp. 70–76.
[26] M. A. Akbar, K. Smolander, S. Mahmood, and A. Alsanad, "Toward successful DevSecOps in software development organizations: A decision-making framework," *Inf. Softw. Technol.*, vol. 147, Jul. 2022, Art. no. 106894.
[27] M. Goldschmidt and M. McKinnon, "DevSecOps—Agility with security," Sense Secur., Australia, Tech. Rep. EBSE-2007-01, p. 44, 2016.
[28] (2021). *The State of DevOps Report 2021*. Puppet. [Online]. Available: https://puppet.com/resources/report/2021-state-of-devops-report
[29] (2021). *A Maturing DevSecOps Landscape, GitLab*. [Online]. Available: https://learn.gitlab.com/c/2021-devsecops-report?x=u5RjB_
[30] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting DevSecOps: A systematic review," *Inf. Softw. Technol.*, vol. 141, Jan. 2022, Art. no. 106700.
[31] N. Tomas, J. Li, and H. Huang, "An empirical study on culture, automation, measurement, and sharing of DevSecOps," in *Proc. Int. Conf. Cyber Secur. Protection Digit. Services (Cyber Security)*, Jun. 2019, pp. 1–8.
[32] R. Mao, H. Zhang, Q. Dai, H. Huang, G. Rong, H. Shen, L. Chen, and K. Lu, "Preliminary findings about DevSecOps from grey literature," in *Proc. IEEE 20th Int. Conf. Softw. Qual., Rel. Secur. (QRS)*, Dec. 2020, pp. 450–457.
[33] L. Prates, J. Faustino, M. M. D. Silva, and R. Pereira, "DevSecOps metrics," in *Proc. EuroSymp. Syst. Anal. Design*. Cham, Switzerland: Springer, Jan. 2019, pp. 77–90.
[34] W. Mallouli, A. Cavalli, A. Bagnato, and E. M. de Oca, "Metrics-driven DevSecOps," in *Proc. 15th Int. Conf. Softw. Technol.*, 2020, pp. 228–233.
[35] V. Garousi, M. Felderer, and M. V. Mäntylä, "Guidelines for including grey literature and conducting multivocal literature reviews in software engineering," *Inf. Softw. Technol.*, vol. 106, pp. 101–121, Feb. 2019.
[36] V. J. White, J. M. Glanville, C. Lefebvre, and T. A. Sheldon, "A statistical approach to designing search filters to find systematic reviews: Objectivity enhances accuracy," *J. Inf. Sci.*, vol. 27, no. 6, pp. 357–370, Dec. 2001.
[37] H. Zhang, M. A. Babar, and P. Tell, "Identifying relevant studies in software engineering," *Inf. Softw. Technol.*, vol. 53, no. 6, pp. 625–637, Jun. 2011.
[38] M. Niazi, D. Wilson, and D. Zowghi, "Critical success factors for software process improvement implementation: An empirical study," *Softw. Process, Improvement Pract.*, vol. 11, no. 2, pp. 193–211, Mar. 2006.
[39] L. Chen, M. Ali Babar, and H. Zhang, "Towards an evidence-based understanding of electronic data sources," in *Proc. 14th Int. Conf. Eval. Assessment Softw. Eng. (EASE)*, 2010, pp. 1–4.
[40] W. Afzal, R. Torkar, and R. Feldt, "A systematic review of search-based testing for non-functional system properties," *Inf. Softw. Technol.*, vol. 51, no. 6, pp. 957–976, Jun. 2009.
[41] M. A. Akbar, S. Mahmood, A. Alsanad, A. A. Alsanad, A. Gumaei, and S. F. Qadri, "A multivocal study to improve the implementation of global requirements change management process: A client-vendor prospective," *J. Softw., Evol. Process*, vol. 32, no. 8, p. 2252, Aug. 2020.
[42] G. T. G. Neto, W. B. Santos, P. T. Endo, and R. A. A. Fagundes, "Multivocal literature reviews in software engineering: Preliminary findings from a tertiary study," in *Proc. ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*, Sep. 2019, pp. 1–6.
[43] M. A. Akbar, S. Mahmood, C. Meshram, A. Alsanad, A. Gumaei, and S. A. AlQahtani, "Barriers of managing cloud outsource software development projects: A multivocal study," *Multimedia Tools Appl.*, vol. 81, pp. 35571–35594, Oct. 2021.
[44] M. Niazi, S. Mahmood, M. Alshayeb, A. M. Qureshi, K. Faisal, and N. Cerpa, "Toward successful project management in global software development," *Int. J. Project Manage.*, vol. 34, no. 8, pp. 1553–1567, Nov. 2016.
[45] A. A. Khan, J. Keung, S. Hussain, M. Niazi, and S. Kieffer, "Systematic literature study for dimensional classification of success factors affecting process improvement in global software development: Client–vendor perspective," *IET Softw.*, vol. 12, no. 4, pp. 333–344, Aug. 2018.
[46] S. U. Khan, M. Niazi, and R. Ahmad, "Barriers in the selection of offshore software development outsourcing vendors: An exploratory study using a systematic literature review," *Inf. Softw. Technol.*, vol. 53, no. 7, pp. 693–706, Jul. 2011.
[47] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Evidence-Based Softw. Eng. (EBSE), Keele Univ., Durham Univ., U.K., Tech. Rep. EBSE-2007-01, 2007.
[48] J. M. Corbin and A. Strauss, "Grounded theory research: Procedures, canons, and evaluative criteria," *Qualitative Sociol.*, vol. 13, no. 1, pp. 3–21, 1990.
[49] K. A. Hallgren, "Computing inter-rater reliability for observational data: An overview and tutorial," *Tuts. Quant. Methods Psychol.*, vol. 8, no. 1, pp. 23–34, Feb. 2012.
[50] G. Kim, "Ultimate guide to CAMS model in DevOps," Tripwire, Sweden, Tech. Rep. 1323, 2020.
[51] T. H.-C. Hsu, *Practical Security Automation and Testing: Tools and Techniques for Automated Security Scanning and Testing in Devsecops*. Birmingham, U.K.: Packt, 2019.
[52] M. Callanan and A. Spillane, "DevOps: Making it easy to do the right thing," *IEEE Softw.*, vol. 33, no. 3, pp. 53–59, May 2016.
[53] B. Kitchenham and S. L. Pfleeger, "Principles of survey research part 6: Data analysis," *ACM SIGSOFT Softw. Eng. Notes*, vol. 28, no. 2, pp. 24–27, Mar. 2003.
[54] M. A. Akbar, J. Sang, A. A. Khan, and S. Hussain, "Investigation of the requirements change management challenges in the domain of global software development," *J. Softw., Evol. Process*, vol. 31, no. 10, p. 2207, Oct. 2019.
[55] M. A. Akbar, J. Sang, A. A. Khan, S. Mahmood, S. F. Qadri, H. Hu, and H. Xiang, "Success factors influencing requirements change management process in global software development," *J. Comput. Lang.*, vol. 51, pp. 112–130, Apr. 2019.
[56] A. A. Khan, J. W. Keung, and M. Abdullah-Al-Wadud, "SPIIMM: Toward a model for software process improvement implementation and management in global software development," *IEEE Access*, vol. 5, pp. 13720–13741, 2017.
[57] A. A. Khan, J. Keung, M. Niazi, S. Hussain, and A. Ahmad, "Systematic literature review and empirical investigation of barriers to process improvement in global software development: Client–vendor perspective," *Inf. Softw. Technol.*, vol. 87, pp. 180–205, Jul. 2017.

[58] C. Noy, "Sampling knowledge: The hermeneutics of snowball sampling in qualitative research," *Int. J. Social Res. Methodol.*, vol. 11, no. 4, pp. 327–344, Oct. 2008.
[59] S. Easterbrook, J. Singer, M.-A. Storey, and D. Damian, "Selecting empirical methods for software engineering research," in *Selecting Empirical Methods for Software Engineering Research*. London, U.K.: Springer, 2008, pp. 285–311.
[60] K. Finstad, "Response interpolation and scale sensitivity: Evidence against 5-point scales," *J. Usability Stud.*, vol. 5, no. 3, pp. 104–110, May 2010.
[61] A. S. Campanelli, R. D. Camilo, and F. S. Parreiras, "The impact of tailoring criteria on agile practices adoption: A survey with novice agile practitioners in Brazil," *J. Syst. Softw.*, vol. 137, pp. 366–379, Mar. 2018.
[62] J. F. Hair, C. M. Ringle, and M. Sarstedt, "PLS-SEM: Indeed a silver bullet," *J. Marketing Theory Pract.*, vol. 19, no. 2, pp. 139–152, Apr. 2011.
[63] D. Russo and K.-J. Stol, "PLS-SEM for software engineering research: An introduction and survey," *ACM Comput. Surveys*, vol. 54, no. 4, pp. 1–38, May 2022.
[64] H. Wold, "Soft modelling: The basic design and some extensions, systems under indirect observation," *Syst. Under Indirect Observ., II*, vol. 2, pp. 36–37, Jan. 1982.
[65] Y. Wang, "Building the linkage between project managers' personality and success of software projects," in *Proc. 3rd Int. Symp. Empirical Softw. Eng. Meas.*, Oct. 2009, pp. 410–413.
[66] M. Senapathi and A. Srinivasan, "An empirical investigation of the factors affecting agile usage," in *Proc. 18th Int. Conf. Eval. Assessment Softw. Eng.*, May 2014, pp. 1–10.
[67] S. Bahl and O. P. Wali, "Perceived significance of information security governance to predict the information security service quality in software service industry: An empirical analysis," *Inf. Manage. Comput. Secur.*, vol. 22, no. 1, pp. 2–23, 2014.
[68] R. Govindaraju, A. Bramagara, L. Gondodiwiryo, and T. Simatupang, "Requirement volatility, standardization and knowledge integration in software projects: An empirical analysis on outsourced IS development projects," *J. ICT Res. Appl.*, vol. 9, no. 1, pp. 68–87, Jun. 2015.
[69] L. Ramanathan and S. Krishnan, "An empirical investigation into the adoption of open source software in information technology outsourcing organizations," *J. Syst. Inf. Technol.*, vol. 17, no. 2, pp. 167–192, 2015.
[70] W. W. Chin, "How to write up and report PLS analyses," in *Esposito Vinzi*, V. Chin, W. W. J. Henseler, and H. Wang, Eds., Berlin, Germany: Springer, 2010.
[71] D. George, *SPSS for Windows Step by Step: A Simple Study Guide and Reference, 17.0 Update, 10/E*. London, U.K.: Pearson, 2011.
[72] J. Henseler, C. M. Ringle, and R. R. Sinkovics, "The use of partial least squares path modeling in international marketing," in *New Challenges to International Marketing*. Bingley, U.K.: Emerald Group Publishing Limited, 2009.
[73] R. M. O'brien, "A caution regarding rules of thumb for variance inflation factors," *Qual. Quantity*, vol. 41, no. 5, pp. 673–690, Sep. 2007.
[74] C. Fornell and D. F. Larcker, "Evaluating structural equation models with unobservable variables and measurement error," *J. Marketing Res.*, vol. 18, no. 1, pp. 39–50, Feb. 1981.
[75] K. K. K.-K. Wong, "Partial least squares structural equation modeling (PLS-SEM) techniques using SmartPLS," *Mark. Bull.*, vol. 24, no. 1, pp. 1–32, 2013.
[76] T. K. Dijkstra and J. Henseler, "Consistent and asymptotically normal PLS estimators for linear structural equations," *Comput. Statist. Data Anal.*, vol. 81, pp. 10–23, Jan. 2015.
[77] K. Casey, "The enterprises project," Experts Report, Raleigh, NC, USA, Tech. Rep. 6314, Jun. 2018.
[78] M. Chiodi, "How to create a DevSecOps culture," Santa Clara, CA, USA, Jun. 2020.
[79] G. Robinson, "Overcoming challenges to automating DevSecOps," MediaOps, Boca Raton, FL, USA, 2021.
[80] O. Díaz, M. Muñoz, and J. Mejía, "Responsive infrastructure with cybersecurity for automated high availability DevSecOps processes," in *Proc. 8th Int. Conf. Softw. Process Improvement (CIMPS)*, Oct. 2019, pp. 1–9.
[81] J. Morales, R. Turner, S. Miller, P. Capell, P. Place, and D. J. Shepard, "Guide to implementing devsecops for a system of systems in highly regulated environments," Softw. Eng. Inst., Carnegie-Mellon Univ. Pittsburgh, PA, USA, Tech. Rep. CMU/SEI-2020-TR-002, 2020.
[82] R. W. Stoddard, "Analytic tools/tips for your agile DevSecOps measurement toolkit," Softw. Eng. Inst., Carnegie-Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU/SEI-2019-TR-004, 2019.
[83] M. Niazi, S. Mahmood, M. Alshayeb, M. R. Riaz, K. Faisal, N. Cerpa, S. U. Khan, and I. Richardson, "Challenges of project management in global software development: A client-vendor analysis," *Inf. Softw. Technol.*, vol. 80, pp. 1–19, Dec. 2016.
[84] A. Saltan and K. Smolander, "Bridging the state-of-the-art and the state-of-the-practice of SaaS pricing: A multivocal literature review," *Inf. Softw. Technol.*, vol. 133, May 2021, Art. no. 106510.
[85] M. Csikszentmihalyi and R. Larson, "Validity and reliability of the experience-sampling method," in *Flow and the Foundations of Positive Psychology*. Berlin, Germany: Springer, 2014, pp. 35–54.
[86] R. C. MacCallum, K. F. Widaman, S. Zhang, and S. Hong, "Sample size in factor analysis," *Psychol. Methods*, vol. 4, no. 1, p. 84, 1999.
[87] V. Mohan and L. B. Othmane, "SecDevOps: Is it a marketing buzzword—Mapping research on security in DevOps," in *Proc. 11th Int. Conf. Availability, Rel. Secur. (ARES)*, Aug. 2016, pp. 542–547.
[88] S. Rafi, W. Yu, M. A. Akbar, A. Alsanad, and A. Gumaei, "Prioritization based taxonomy of DevOps security challenges using PROMETHEE," *IEEE Access*, vol. 8, pp. 105426–105446, 2020.

MUHAMMAD AZEEM AKBAR received the Ph.D. degree in software engineering from Chongqing University, China, in January 2019, and the Ph.D. degree from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in January 2021. From January 2021 to August 2021, he was with the LERO-The Irish Software Research Centre, Ireland, as a Senior Researcher, and worked on Huawei's Software Trustworthy Project. He is currently an Associate Professor and a Docent (Adjunct Professor) of software process improvement and management with the Department of Software Engineering, Lappeenranta–Lahti University of Technology, Finland. He is an active researcher in the field of software engineering. He has worked as a principal and co-investigator in several research projects investigating issues related to software process improvements, global software development, and quantum software engineering projects. He is working as the Project Manager of 6GSoft: 6G software for extremely distributed and heterogeneous massive networks of connected devices. He has published more than 150 papers in peer-reviewed journals and international conferences. He actively conducts publication-based research workshops, serves as the guest editor for major software engineering journals, and edits software engineering research books. His research interests include empirical software engineering, blockchain-based software engineering, quantum software engineering, 6G software, ethics of AI and the IoT, software security, global software development, software requirements engineering and change management, DevOps, DevSecOps, and software process improvement in general.

ABEER ABDULAZIZ ALSANAD received the bachelor's degree in computer science from Prince Sultan University, Saudi Arabia, in 2008, and the master's and Ph.D. degrees in information systems from the College of Computer and Information Sciences, King Saud University, Saudi Arabia, in 2011 and 2019, respectively. She is currently an Assistant Professor of information systems with Imam Mohammad Ibn Saud Islamic University, Saudi Arabia. She has worked as a principal and co-investigator in several research projects investigating issues related to software process improvements, global software development, and quantum software engineering projects. She has published several conference and journal papers. Her major research interests include software engineering, requirement engineering, change management, global software development, empirical software engineering, blockchain-based software engineering, quantum software engineering, AI, the IoT, software security, DevOps, and user experience.